

Resumen - Primer Parcial

Ingeniería y Calidad de Software

Resumen unificado a partir del apunte general y de tus notas tomadas en clase. Cuando hay una definición específica de la cátedra, se prioriza esa formulación.

UNIDAD 1 - Ingeniería de Software en contexto

1. Ingeniería de Software

Qué es: Disciplina que se ocupa de todos los aspectos de la producción de software, desde la especificación inicial hasta su mantenimiento y evolución.

Quién lo utiliza / interviene: Equipos y organizaciones que desarrollan, mantienen o gestionan software.

Cómo se utiliza: Aplicando procesos, métodos, prácticas, personas y herramientas para transformar necesidades en un producto de software de calidad.

El software no es solamente código: también incluye documentación, configuraciones, datos y demás artefactos necesarios para definir, construir, ejecutar y mantener el sistema.

2. Proceso y Proceso de Software

Qué es: Un proceso es una secuencia de pasos ejecutados para un propósito dado. Un proceso de software es el conjunto de actividades, métodos, prácticas y transformaciones que las personas usan para desarrollar o mantener software y sus productos asociados.

Quién lo utiliza / interviene: El equipo de desarrollo y la organización.

Cómo se utiliza: El proceso toma ideas/requerimientos como entrada, organiza actividades, personas, recursos y herramientas, y produce un resultado.

- Las personas son el componente central: deben estar capacitadas, tener conocimiento técnico y estar motivadas.
- Las herramientas y equipos facilitan o automatizan actividades.
- Los procedimientos y métodos definen qué hacer y cómo hacerlo.

3. Procesos Empíricos vs. Procesos Definidos

Procesos Empíricos:

- Se basan fuertemente en la experiencia.
- El proceso no está completamente definido de antemano; se adapta.
- Trabajan con ciclos cortos de implementación y retroalimentación.
- La experiencia obtenida en un contexto no es directamente extrapolable a otros proyectos.
- Se asocian al ciclo de vida iterativo/incremental.

Procesos Definidos:

- Tienen una serie de pasos establecidos.
- Buscan repetir el mismo proceso y obtener resultados previsibles.

- La experiencia puede reutilizarse con mayor facilidad.
- Pueden combinarse con distintos ciclos de vida.

4. Ciclo de vida

Qué es: Serie de pasos a través de los cuales un producto o proyecto progresa en el tiempo. Es una representación de un proceso desde una perspectiva particular.

Quién lo utiliza / interviene: El equipo y quienes gestionan el proyecto.

Cómo se utiliza: Permite decidir cómo se encarará el proceso: qué orden tendrán las actividades y cuánto se hará en cada etapa.

Tipos básicos:

- Secuencial / Cascada.
- Iterativo-Incremental.
- Recursivo / Espiral.

5. Modelo Secuencial / Cascada

Organiza el trabajo en etapas consecutivas. La versión entregable del producto aparece recién hacia el final.

Ventajas:

- Estructura clara y fácil de seguir.
- Facilita planificación y control cuando existe alta certeza.
- Adecuado con requerimientos claros, estables y exhaustivos.

Desventajas:

- Dificultad para absorber cambios.
- El cliente ve el producto funcional tarde.
- Los defectos tardíos pueden exigir mucho retrabajo.
- No maneja bien la incertidumbre.

6. Modelo Iterativo-Incremental

El producto se construye en sucesivas iteraciones. Cada iteración genera un incremento funcional y agrega valor sobre lo anterior.

- Permite feedback frecuente.
- Facilita adaptación de requerimientos.
- Entrega valor más temprano.
- Es apropiado para contextos inciertos y cambiantes.
- Es el ciclo de vida asociado a los procesos empíricos y a Agile.

7. Modelo Recursivo / Espiral

Organiza el desarrollo en ciclos orientados a identificar y reducir riesgos.

- Determinar objetivos.
- Identificar y resolver riesgos.
- Desarrollar y probar.
- Planificar la próxima iteración.

Elección rápida: Secuencial = alta certeza | Iterativo-Incremental = cambio/incertidumbre | Espiral = riesgo alto.

8. Proyecto, Producto y Proceso

Qué es: Proceso = conjunto de actividades; Proyecto = medio temporal para administrar recursos y alcanzar un objetivo único; Producto = software resultante, que puede evolucionar a través de versiones.

Quién lo utiliza / interviene: El equipo del proyecto, roles de gestión y stakeholders.

Cómo se utiliza: El proceso organiza el trabajo; el proyecto aplica ese proceso con recursos/personas; el producto es el resultado.

Características del proyecto:

- Tiene un objetivo único.
- Tiene temporalidad: inicio y fin.
- Los objetivos deben ser claros y alcanzables.
- Contiene tareas interrelacionadas y recursos.

9. Administración de Proyectos - enfoque tradicional

Busca lograr el objetivo del proyecto organizando personas y recursos y realizando seguimiento y control.

- Planificación.
- Seguimiento.
- Control.

Triple restricción:

- Alcance: características/performance del producto o servicio.
- Tiempo: fecha comprometida.
- Costo: recursos y personas.

Plan de proyecto: responde qué, cuándo, cómo y quién hará el trabajo.

- Definición del alcance.
- Definición del proceso y ciclo de vida.
- Estimación.
- Gestión de riesgos.
- Asignación de recursos.
- Programación.
- Controles.
- Métricas.

10. Alcance del Producto vs. Alcance del Proyecto

Alcance del Producto: todas las características que puede incluir el producto/servicio. Se mide en requerimientos.

Alcance del Proyecto: todo el trabajo necesario para entregar ese producto. Se mide en tareas.

11. Estimaciones de Software - enfoque tradicional

- Tamaño: cuánto producto se construirá.
- Esfuerzo: horas-persona lineales.
- Calendario: cuándo estará listo.
- Costo.
- Recursos críticos.

12. Crisis del Software

Qué es: Nombre dado al conjunto de problemas históricos para construir software confiable, mantenible, verificable y dentro de costos y plazos razonables.

Quién lo utiliza / interviene: Es un concepto de Ingeniería de Software usado para explicar por qué fue necesario profesionalizar la disciplina.

Cómo se utiliza: Sirve para analizar causas de fracaso y justificar el uso de procesos, métodos, gestión y disciplinas

de soporte.

- El hardware evolucionó muy rápido y habilitó sistemas más grandes.
- El software siguió siendo una actividad humana, abstracta e intangible.
- Crecieron la demanda y la complejidad.
- Los requerimientos cambian.
- La falta de métodos de ingeniería produjo sobrecostos, demoras y baja confiabilidad.

13. Disciplinas de la Ingeniería de Software

Técnicas: requerimientos, análisis, diseño, implementación, pruebas, despliegue, documentación y capacitación.

Gestión: planificación, monitoreo y control.

Soporte: actividades transversales como SCM, métricas y aseguramiento de calidad.

Técnicas = construir | Gestión = organizar/controlar | Soporte = preservar integridad/calidad.

14. Ejemplos de proyectos de software fallidos y exitosos

La cátedra utiliza estos ejemplos para mostrar que el éxito o fracaso de un proyecto no depende solo de programar bien, sino también de requerimientos, gestión, proceso, riesgos, personas, comunicación y control. Para el parcial, lo importante es poder relacionar un caso con las causas típicas de éxito o fracaso.

15. No Silver Bullet - Fred Brooks

Qué es: Es un paper/ensayo académico de Fred Brooks. NO es una técnica, metodología ni framework. Plantea que no existe una única 'bala de plata' capaz de producir por sí sola una mejora revolucionaria en productividad, confiabilidad y simplicidad del desarrollo de software.

Quién lo utiliza / interviene: Lo utiliza la Ingeniería de Software como marco conceptual para analizar la naturaleza de los problemas del desarrollo.

Cómo se utiliza: No se 'aplica' como un método paso a paso. Se usa para distinguir qué dificultades pueden reducirse con herramientas y cuáles son inherentes al software.

Dificultades esenciales:

- Complejidad: es inherente al software.
- Conformidad: el software debe adaptarse a necesidades y restricciones externas.
- Mutabilidad: el software cambia continuamente.
- Invisibilidad: no tiene una representación física natural.

Dificultades accidentales: dependen de las herramientas/técnicas usadas para construir software y pueden reducirse con mejores lenguajes, herramientas y entornos.

Respuesta de parcial: No Silver Bullet es un paper de Fred Brooks; su tesis es que no existe una solución mágica capaz de eliminar las dificultades esenciales del software.

UNIDAD 2 - Agile, Requerimientos, Producto y Scrum

16. Filosofía Ágil y Manifiesto Ágil

Qué es: Filosofía de desarrollo basada en procesos empíricos, ciclos de retroalimentación cortos y adaptación continua.

Quién lo utiliza / interviene: Equipos de desarrollo y organizaciones que trabajan en contextos complejos/cambiantes.

Cómo se utiliza: Priorizando valor, software funcionando, colaboración con el cliente y respuesta al cambio.

Tres pilares del empirismo:

- Inspección.
- Adaptación.
- Transparencia.

Valores del Manifiesto Ágil:

- Individuos e interacciones por sobre procesos y herramientas.
- Software funcionando por sobre documentación extensiva.
- Colaboración con el cliente por sobre negociación contractual.
- Responder al cambio por sobre seguir un plan.

12 principios:

- Satisfacer al cliente.
- Aceptar cambios de requisitos.
- Entregar software funcional frecuentemente.
- Negocio y desarrollo trabajan juntos.
- Trabajar con individuos motivados.
- Favorecer conversación cara a cara.
- Software funcionando como medida principal de progreso.
- Desarrollo sostenible.
- Atención continua a excelencia técnica y buen diseño.
- Simplicidad.
- Equipos auto-organizados/autogestionados.
- Reflexión y ajuste periódico.

17. Gestión Ágil de Requerimientos

Qué es: Forma de gestionar requerimientos en la que se priorizan, refinan y detallan progresivamente en lugar de definirlos todos exhaustivamente al inicio.

Quién lo utiliza / interviene: Product Owner, usuarios/negocio y equipo de desarrollo.

Cómo se utiliza: Priorizando por valor y trabajando Just in Time: analizar cuando se necesita, no antes.

- Usar el valor para construir el producto correcto.
- Usar historias/modelos para mostrar qué construir.
- Definir solo lo suficiente.
- Los cambios pueden convertirse en ventaja competitiva.
- Agile se enfoca especialmente en requerimientos de usuario y de negocio.
- Tiempo y recursos se mantienen fijos; el alcance se adapta.

18. User Stories

Qué es: Descripción corta de una necesidad que tiene un usuario del software.

Quién lo utiliza / interviene: Las escribe/gestiona el Product Owner con participación del negocio y conversación con Developers.

Cómo se utiliza: Se incorporan al Product Backlog, se priorizan, se conversan y luego se implementan como porciones verticales de valor.

CCC - Partes de una User Story:

- Tarjeta: Como [rol], yo puedo/quiero [actividad], de forma tal que [valor de negocio].
- Conversación: permite entender y detallar la necesidad.
- Confirmación: criterios/pruebas que permiten demostrar que se cumplió.

Criterios de aceptación: según la cátedra, definen los límites de una US; deben ser verificables y expresan una intención, no una solución.

Pruebas de aceptación: se derivan de los criterios y describen escenarios concretos.

Las User Stories también funcionan como:

- Necesidad de usuario.
- Descripción del producto.
- Ítem de planificación.
- Token/recordatorio para una conversación.
- Mecanismo para diferir una conversación.

Porciones verticales: una US puede incluir algo de front, backend y base de datos para entregar valor completo.

Niveles de abstracción: Tema -> Épica -> User Story.

19. Definition of Ready - según la cátedra

Qué es: Criterio para determinar si una User Story entra al Product Backlog. Como mínimo debe cumplir el modelo INVEST.

Quién lo utiliza / interviene: El equipo y el Product Owner.

Cómo se utiliza: Se usa como filtro de calidad previo: una US que no cumple el mínimo acordado no debería incorporarse como ítem listo.

INVEST:

- I - Independent: implementable/calendarizable con independencia.
- N - Negotiable: define el qué, no fija el cómo.
- V - Valuable: aporta valor al cliente/negocio.
- E - Estimable: puede estimarse.
- S - Small: suficientemente pequeña para una iteración.
- T - Testable: puede demostrarse que fue implementada.

20. Definition of Done - según la cátedra

Qué es: Criterio para determinar si el incremento puede entrar al producto y ser aceptado por el Product Owner. Expresa la calidad y las características que debe cumplir el producto.

Quién lo utiliza / interviene: Todo el Scrum Team; el Product Owner utiliza el resultado para aceptar el incremento.

Cómo se utiliza: Se verifica al finalizar el trabajo. Si no cumple la DoD, el incremento no se considera terminado/aceptable.

Diferencia de la cátedra: DoR = criterio para entrar al Product Backlog | DoD = criterio para entrar al producto y ser aceptado por el Product Owner.

21. Gestión de Producto - MVP

Qué es: MVP (Minimum Viable Product) es el conjunto mínimo de funcionalidades de un producto necesario para validar una hipótesis.

Quién lo utiliza / interviene: Equipo de producto, Product Owner, negocio y usuarios objetivo.

Cómo se utiliza: Se construye con la menor inversión razonable de tiempo, dinero y esfuerzo para aprender si la

hipótesis tiene sentido antes de invertir más.

- Se usa cuando hay una idea de producto con mucha incertidumbre.
- Puede ser un producto pequeño, prototipo, video u otra forma válida de probar la hipótesis.
- Pensar en grande y comenzar de a poco.

Pasos para definirlo:

- 1. Definir la hipótesis.
- 2. Definir descripción/alcance.
- 3. Identificar las User Stories que forman parte del MVP.
- 4. Identificar explícitamente las User Stories fuera de alcance.

22. Estimaciones

Una estimación tiene un valor probabilístico: no es exacta. Estimar y planificar no son lo mismo.

Métodos generales:

- Datos históricos.
- Juicio experto individual o grupal.
- Analogía.
- Basados en recursos.
- Basados en mercado.
- Basados en componentes/producto/proceso.
- Métodos algorítmicos.

23. Estimaciones Ágiles

Qué es: Estimación relativa del tamaño de features/User Stories.

Quién lo utiliza / interviene: Los Developers/equipo que realizará el trabajo.

Cómo se utiliza: Comparando historias entre sí mediante una escala relativa, no mediante tiempo absoluto.

Story Point: medida específica del equipo que integra complejidad, esfuerzo e incertidumbre.

- Complejidad: dificultad y relaciones internas de la historia.
- Esfuerzo: cantidad de trabajo requerido.
- Incertidumbre: información técnica o de negocio que falta.

Escalas posibles: números, talles de remera, potencias de 2 o Fibonacci.

24. US canónica y Poker Estimation

Qué es: La US canónica es la historia elegida como patrón de referencia para comparar el tamaño de las demás.

Quién lo utiliza / interviene: Todo el equipo que participa en la estimación.

Cómo se utiliza: Al inicio de la sesión se elige una historia básica/entendible y se acuerda su puntaje. Luego las demás se estiman comparándolas con ella.

Poker Estimation:

- Se presenta la US.
- El equipo conversa sobre desafíos, tareas e incertidumbre.
- Cada integrante asigna un puntaje.
- Si hay acuerdo, se toma el valor.
- Si no hay acuerdo, se debate y se vuelve a estimar.

La US canónica se elige respecto del conjunto de historias que se está estimando; no está limitada solo a las historias del MVP.

25. Velocidad

Qué es: Métrica del progreso del equipo: cantidad de Story Points de User Stories completas en una iteración.

Quién lo utiliza / interviene: El Scrum Team la observa para planificar y obtener previsibilidad.

Cómo se utiliza: Se calcula al final del Sprint sumando solo los Story Points de historias completas; las parcialmente terminadas no cuentan.

Fórmulas útiles de clase:

- Velocidad promedio = Story Points completados / cantidad de Sprints observados.
- Duración aproximada = Story Points totales pendientes / Velocidad.

SCRUM

26. Qué es Scrum

Qué es: Marco de trabajo liviano que ayuda a personas, equipos y organizaciones a generar valor mediante soluciones adaptativas para problemas complejos.

Quién lo utiliza / interviene: Scrum Team: Product Owner, Scrum Master y Developers.

Cómo se utiliza: Organiza el trabajo en Sprints y utiliza empirismo, enfoque iterativo-incremental e inteligencia colectiva.

- No es una metodología rígida.
- Puede convivir con distintos procesos, técnicas y métodos.
- Sus bases son empirismo, iterativo-incremental y reducción de desperdicios.

27. Valores de Scrum

- Compromiso.
- Enfoque.
- Apertura.
- Respeto.
- Coraje.

28. Scrum Team

Responsable de crear un incremento valioso y útil en cada Sprint. Es multifuncional, autogestionado, sin jerarquías y generalmente de 10 personas o menos.

Scrum Master:

- No es el líder de proyecto.
- Responsable de establecer Scrum según la guía.
- Sirve al equipo, al Product Owner y a la organización.

Product Owner:

- Maximiza el valor del producto.
- Gestiona eficazmente el Product Backlog.
- Sus decisiones deben ser respetadas.
- Es una sola persona.

Developers:

- Crean el Sprint Backlog.
- Construyen el incremento.
- Mantienen la calidad según la DoD.
- Adaptan su plan al Sprint Goal.
- Se responsabilizan mutuamente.

29. Eventos / Ceremonias y Timebox

Timebox: tiempo fijo definido para cada evento.

Sprint: ciclo de trabajo con un objetivo; dura 1 mes o menos. Al terminar, comienza otro inmediatamente.

Sprint Planning: máximo 8 horas para un Sprint de un mes.

- ¿Por qué este Sprint es valioso?
- ¿Qué se puede hacer en este Sprint?
- ¿Cómo se realizará el trabajo elegido?

Daily Scrum: 15 minutos, diaria, mismo tiempo/lugar según las notas de clase.

- ¿Hay impedimentos?
- ¿Qué hice ayer?
- ¿Qué haré hoy?

Sprint Review: máximo 4 horas para un Sprint de un mes. Inspecciona el resultado, descubre cambios y reordena información/prioridades.

Sprint Retrospective: máximo 3 horas. Última actividad al cerrar el Sprint; analiza qué salió bien, problemas y cómo mejorar.

Refinamiento del Product Backlog: actividad continua; en tus notas figura como referencia aproximadamente 10% del tiempo del Sprint.

30. Artefactos de Scrum y compromisos

Artefacto	Qué es / contenido	Compromiso
Product Backlog	Lista emergente y ordenada de lo que se necesita para mejorar el producto.	Product Goal
Sprint Backlog	Sprint Goal + PBIs seleccionados + plan para crear el incremento.	Sprint Goal
Product Increment	Paso concreto, aditivo y valioso hacia el objetivo del producto.	Definition of Done

31. Sprint Backlog - por qué / qué / cómo

- Por qué se hace el Sprint -> Sprint Goal / valor del Sprint.
- Qué se realizará -> PBIs seleccionados.
- Cómo se realizará -> plan de los Developers.

32. Capacidad del equipo

Qué es: Cantidad de trabajo disponible/comprometible para un Sprint.

Quién lo utiliza / interviene: El Scrum Team durante Sprint Planning.

Cómo se utiliza: Se estima al inicio. Según tus notas: horas de trabajo disponibles por día x días disponibles de la iteración; también puede expresarse en Story Points.

33. Herramientas de Scrum

Taskboard / tablero:

- Story.
- To Do.
- In Process.
- To Verify.
- Done.

Las historias se estiman en Story Points; las tareas pueden manejarse en horas ideales. El tablero debe tener una granularidad suficiente para hacer visible el avance.

Sprint Burn-Down Chart: visualiza el trabajo pendiente/quemado durante el Sprint. Eje vertical: Story Points; eje horizontal: días. Se actualiza a medida que se completa trabajo.

Sprint Burn-Up Chart: muestra trabajo completado/acumulado a lo largo del tiempo.

34. Métricas ágiles que sí entran

RTS - Running Tested Features:

- Cantidad de características que están funcionando y fueron desarrolladas.
- Se calcula.
- Representa producto efectivamente funcionando.

Capacidad:

- Horas disponibles por día x días disponibles de iteración, o equivalente acordado.
- Se estima al principio del Sprint.

Velocidad:

- Story Points de US completas.
- Se calcula al final del Sprint.
- Mide el progreso del equipo.

Para este parcial, estudiar específicamente: RTS + Capacidad + Velocidad.

35. Múltiples niveles de planificación

- Estrategia.
- Portfolio.
- Producto.
- Release.
- Iteración / Sprint.
- Día / Daily.

Si cambian las circunstancias o condiciones sobre las cuales se estimó/planificó, debe revisarse la planificación y, si corresponde, volver a estimar.

UNIDAD 3 - Gestión de Software como Producto: SCM

36. SCM - Software Configuration Management

Qué es: Disciplina de soporte transversal que permite identificar, organizar, controlar y registrar la evolución de los ítems de software para mantener la integridad y trazabilidad del producto.

Quién lo utiliza / interviene: Equipo de desarrollo, responsables de configuración, líderes/roles del proyecto, comité de cambios y auditores cuando corresponde.

Cómo se utiliza: Controlando versiones, líneas base, repositorio, cambios, auditorías e informes de estado durante el ciclo de vida.

- Evita pérdida de componentes.
- Evita pérdida de cambios/versiones.
- Reduce regresiones y doble mantenimiento.
- Evita superposición de cambios.
- Controla cambios no validados.

37. Ítem de Configuración (IC)

Qué es: Elemento del producto/proyecto que se decide administrar bajo control de configuración.

Quién lo utiliza / interviene: El equipo de desarrollo y quien planifica SCM.

Cómo se utiliza: Se identifica de forma unívoca, se versiona, se ubica en el repositorio y se mantiene su historia.

Ejemplos: código fuente, ERS, arquitectura, casos de prueba, base de datos, manuales, planes y reportes.

Clasificación por duración:

- Ítems de producto: se mantienen mientras exista el producto.
- Ítems de proyecto: duran lo que dura el proyecto.
- Ítems de iteración: duran lo que dura una iteración.

38. Versión, Variante y Configuración

Versión: estado particular de un IC en un momento determinado.

Variante: alternativa que evoluciona separadamente para satisfacer diferencias, por ejemplo plataforma/sistema operativo.

Configuración: conjunto de todos los IC con sus versiones específicas en un momento determinado; una 'foto' del software.

39. Repositorio

Qué es: Contenedor estructurado de ítems de configuración que conserva su historia, atributos y relaciones.

Quién lo utiliza / interviene: Equipo y responsables de configuración.

Cómo se utiliza: Se utiliza para almacenar IC, controlar versiones, gestionar acceso/backups y analizar impacto de cambios.

- Centralizado: un servidor concentra archivos/versiones.
- Descentralizado: cada cliente posee una copia completa.

40. Release, Branch y Merge

Release: entrega de un sistema liberada para uso de clientes/usuarios.

Branch: conjunto de IC/versiones que bifurca el desarrollo para experimentación, corrección de errores o plataformas diferentes.

Merge: integración de ramas para combinar configuraciones/cambios.

41. Línea Base

Qué es: Configuración de software formalmente revisada y aprobada que sirve como punto de referencia para desarrollos posteriores.

Quién lo utiliza / interviene: Equipo, responsables de configuración y Comité de Control de Cambios.

Cómo se utiliza: Se identifica de manera única (por ejemplo con tags). Una vez establecida, no se modifica informalmente: cualquier cambio debe pasar por control formal.

- Sirve como punto de referencia.
- Permite rollback.
- Permite saber qué versión se puso en producción.
- Permite reproducir un estado estable anterior.

Tipos:

- Operacional: producto ejecutable que pasó controles definidos.
- De especificación/documentación: requerimientos, diseño u otros documentos antes de disponer de código ejecutable.

42. Planificación de SCM

Debe realizarse tempranamente y definir:

- Reglas de nombrado/versionado.
- Herramientas.
- Roles e integrantes del Comité de Control de Cambios.
- Procedimiento formal de cambios.
- Proceso de auditoría.
- Estructura del repositorio.
- Registros a mantener.
- Documentos/productos administrados.

43. Identificación de Ítems de Configuración

Qué es: Actividad de SCM que determina qué elementos se controlarán y cómo se identificarán.

Quién lo utiliza / interviene: Equipo y responsables de configuración.

Cómo se utiliza: Asignando identificadores únicos, reglas de nombrado/versionado, estructura de repositorio, ubicación y pertenencia a líneas base.

- Identificación unívoca.
- Convenciones de nombrado.
- Definición de estructura del repositorio.
- Ubicación dentro del repositorio.
- Definición de componentes de la línea base.

44. Gestión / Control de Cambios

Qué es: Proceso formal para evaluar, autorizar, ejecutar y comunicar modificaciones sobre una línea base.

Quién lo utiliza / interviene: Comité de Control de Cambios, integrado según el cambio por interesados como líder, analista, arquitecto y/o cliente.

Cómo se utiliza: Una propuesta se analiza por impacto; si se aprueba se genera una orden, se implementa, se revisa la nueva línea base y se notifica a los interesados.

Etapas:

- 1. Recibir propuesta de cambio sobre una línea base.
- 2. Analizar impacto técnico, recursos, efectos secundarios, funcionalidad y arquitectura.
- 3. Si se autoriza, emitir orden de cambio con restricciones y criterios de revisión/auditoría.
- 4. Implementar el cambio y revisar/aprobar la nueva línea base.
- 5. Notificar el cambio a todos los interesados.

El control de cambios también permite mantener trazabilidad entre IC y saber qué elementos resultan afectados.

45. Auditorías de Configuración

Qué es: Controles realizados sobre una línea base para verificar integridad y cumplimiento del plan de gestión de configuración.

Quién lo utiliza / interviene: Un auditor externo al equipo, independiente y objetivo.

Cómo se utiliza: Se audita una línea base durante la construcción del producto para detectar desvíos antes de que generen mayor costo.

Condiciones del auditor:

- Objetivo: independencia de criterio.
- Independiente: no depende jerárquica, funcional ni salarialmente de lo auditado.

46. Auditoría Física

Qué es: Auditoría que verifica primero la consistencia física/documental de los IC de la línea base respecto del plan de SCM.

Quién lo utiliza / interviene: Auditor de configuración.

Cómo se utiliza: Comprueba ubicación del repositorio, esquema de nombrado, ubicación de IC y consistencia entre documentación y código.

- Se realiza antes de la auditoría funcional.
- Audita una línea base.
- Hace verificación.
- Si falla, no se realiza la auditoría funcional.

47. Auditoría Funcional

Qué es: Auditoría que verifica la trazabilidad y que la funcionalidad/performance real de los IC sea consistente con la especificación de requerimientos.

Quién lo utiliza / interviene: Auditor de configuración.

Cómo se utiliza: Contrasta el producto implementado con la ERS y valida que estén implementados todos y solamente los requerimientos correspondientes.

- Requiere que la auditoría física haya resultado satisfactoria.
- Hace validación.
- Revisa trazabilidad entre requerimientos y producto.

48. Verificación vs. Validación en auditorías

Verificación: comprobar que el producto/IC cumple con lo definido en la documentación y plan.

Validación: comprobar que se resolvió el problema correcto y que el producto satisface lo que el cliente necesita.

Auditoría física -> Verificación | Auditoría funcional -> Validación.

49. Trazabilidad

Qué es: Relación entre IC de diferentes niveles de abstracción: requerimiento, análisis/diseño, código y pruebas.

Quién lo utiliza / interviene: Equipo, responsables de cambios y auditores.

Cómo se utiliza: Permite analizar el impacto de cambios y seguir un requerimiento en ambas direcciones.

50. Informe de Estado

Qué es: Mecanismo para registrar cómo evoluciona el sistema y cuál es el estado/ubicación del software respecto de la línea base.

Quién lo utiliza / interviene: Todo el equipo y los interesados que necesitan conocer el estado actualizado de la configuración.

Cómo se utiliza: Registra cambios de las líneas base y comunica la última versión válida para evitar trabajar sobre elementos obsoletos.

- Debe asegurar que la información de cambios llegue a todos.
- Puede automatizarse por el gran volumen de datos.
- El reporte de inventario muestra el contenido y estructura del repositorio.

Preguntas que permite responder:

- ¿Cuál es el estado de un ítem?
- ¿La propuesta de cambio fue aceptada o rechazada?
- ¿Qué versión implementa un cambio aprobado?
- ¿Cuál es la diferencia entre dos versiones?

GUÍA PARA EL PRÁCTICO DEL PARCIAL

51. Qué te pueden pedir

- Definir un MVP.
- Escribir 2 User Stories.
- Estimar esas User Stories.

52. Paso 1 - Definir el MVP

Hipótesis: redactar qué creés que va a ocurrir y con qué usuario.

Alcance: indicar qué problema resuelve la primera versión.

Dentro del MVP: listar las US mínimas necesarias para validar la hipótesis.

Fuera del MVP: dejar explícitas funcionalidades que pueden ser útiles pero no son necesarias para validar ahora.

53. Paso 2 - Escribir 2 User Stories

Plantilla: Como [rol], yo puedo/quiero [actividad], de forma tal que [valor de negocio].

Chequeo mínimo: INVEST, según la Definition of Ready de la cátedra.

Agregar: criterios de aceptación verificables y, si te lo piden, pruebas de aceptación.

54. Paso 3 - Estimar

- Elegir/usar una US canónica.
- Comparar el tamaño relativo de cada historia.
- Considerar complejidad + esfuerzo + incertidumbre.
- Asignar Story Points con la escala acordada, por ejemplo Fibonacci.
- Poder justificar por qué una US vale más o menos que la otra.

55. Ejemplo corto

Hipótesis MVP: creemos que usuarios de una biblioteca utilizarán una función de reserva online si pueden saber inmediatamente si un libro está disponible.

US1: Como lector, quiero consultar la disponibilidad de un libro, de forma tal que sepa si puedo reservarlo. **Estimación:** 3 SP.

US2: Como lector, quiero reservar un libro disponible, de forma tal que pueda retirarlo sin riesgo de que otra persona lo tome. **Estimación:** 5 SP.

Justificación posible: US2 implica más reglas de negocio, actualización de estado y validaciones que US1, por eso su tamaño relativo es mayor.

REPASO ULTRA RÁPIDO

Concepto	Asociación
No Silver Bullet	Paper de Fred Brooks; no es metodología ni técnica
Empírico	Experiencia + inspección + adaptación + transparencia
Secuencial	Alta certeza
Iterativo-Incremental	Cambio, feedback y entregas
Espiral	Riesgos
MVP	Mínimo para validar hipótesis
US	Necesidad de usuario + valor + conversación
DoR - cátedra	Criterio para entrar al Product Backlog; mínimo INVEST

Concepto	Asociación
DoD - cátedra	Criterio para entrar al producto / aceptación del PO
Story Point	Complejidad + esfuerzo + incertidumbre
US canónica	Referencia para estimación relativa
Velocidad	SP completos por Sprint
Capacidad	Trabajo disponible/comprometible del Sprint
RTS	Features funcionando
Scrum	Framework liviano; empirismo + iterativo-incremental
Review	Inspección del resultado/producto
Retrospective	Mejora del trabajo/proceso/equipo
SCM	Integridad + trazabilidad + control de evolución
Línea base	Configuración revisada/aprobada de referencia
Auditoría física	Verificación
Auditoría funcional	Validación
Informe de estado	Registro y comunicación de cambios/configuración